Design Patterns — Refactoring Report

Project: BrainBox
Branch: `feature/design-patterns-refactor`
Author: Gako, Joana Carla D.
Date: April 7, 2026

Refactor 1 — Strategy + Factory Method on Email Service

Original Implementation

EmailService was a single Spring @Service class that handled both SMTP and Resend email delivery internally. A boolean app.debug flag controlled which path ran through an if/else branch inside the sendEmail() method.

Before (Simplified)

```
// BEFORE — EmailService.java
@Service
public class EmailService {
    @Value("${app.debug}")
    private boolean debug;

    public void sendEmail(String to, String subject, String htmlContent) {
        if (debug) {
            sendViaSMTP(to, subject, htmlContent);   // dev path
        } else {
            sendViaResend(to, subject, htmlContent); // prod path
        }
    }
}
```

Problems with the Original

- Mixed responsibilities: One class implemented two distinct delivery mechanisms.
- Conditional coupling: The if(debug) branch evaluated every request rather than once at startup.
- Untestable in isolation: No way to unit-test SMTP logic without also compiling Resend logic.
- Closed to extension: Adding a third provider required modifying EmailService directly, violating OCP.

Applied Design Patterns

Strategy: EmailSender interface with SmtpEmailSender and ResendEmailSender
Factory Method: EmailSenderFactory.create() selects the strategy at startup

After

```java
// EmailSender.java — Strategy interface
public interface EmailSender {
    void send(String to, String subject, String htmlContent);
}

// SmtpEmailSender.java — Concrete Strategy (dev)
public class SmtpEmailSender implements EmailSender {
    @Override
    public void send(String to, String subject, String htmlContent) {
        // MimeMessage setup and send
    }
}

// ResendEmailSender.java — Concrete Strategy (prod)
public class ResendEmailSender implements EmailSender {
    @Override
    public void send(String to, String subject, String htmlContent) {
        // CreateEmailOptions build and send
    }
}

// EmailSenderFactory.java — Factory Method
@Component
public class EmailSenderFactory {
    @PostConstruct
    public void init() { this.emailSender = create(); }
    public EmailSender create() {
        if (debug) return new SmtpEmailSender(mailSender, smtpMailFrom);
        return new ResendEmailSender(new Resend(resendApiKey),
resendFromEmail);
    }
}

// EmailService.java — thin delegator
@Service @RequiredArgsConstructor
public class EmailService {
    private final EmailSenderFactory emailSenderFactory;
    public void sendEmail(String to, String subject, String htmlContent) {
        emailSenderFactory.getSender().send(to, subject, htmlContent);
    }
}
```

Justification

The Strategy pattern was the natural fit because the two delivery mechanisms are entirely interchangeable from the caller's perspective. The Factory Method was paired with it because the selection logic belongs at startup, not at each send call. Together they eliminate conditional branching, give each implementation a single-responsibility class, and allow a new provider to be added by creating one new class without modifying any existing code.

Improvement Brought

Concern	Before	After
Responsibility	One class, two implementations	One class per implementation
Testability	Cannot test SMTP without Resend code	Each strategy tested independently
Extensibility	Modify EmailService to add provider	Add new EmailSender class
Branch at runtime	Every call evaluates if(debug)	Strategy selected once at startup

Refactor 2 — Builder on Notebook Entity Construction

Original Implementation

NotebookService.createNotebook() constructed a Notebook JPA entity with four consecutive setter calls. The category lookup was embedded mid-construction.

```
// BEFORE — NotebookService.createNotebook()
Notebook notebook = new Notebook();
notebook.setTitle(request.getTitle());
notebook.setContent(request.getContent() != null ? request.getContent() : "");
if (request.getCategoryId() != null) {
    Category category = categoryRepository.findById(request.getCategoryId())
            .orElseThrow(() -> new NoSuchElementException("Category not
found"));
    notebook.setCategory(category);
}
notebook.setUser(userService.findById(userId));
```

Problems with the Original

- No construction validation: Nothing prevented a caller from forgetting setUser(), causing a DB constraint failure.
- Construction interleaved with business logic: Category lookup was embedded inside the construction block.
- Repetition risk: Future creation sites would duplicate the same setter sequence and null-guard logic.

Applied Design Pattern

Builder: NotebookBuilder with fluent API and build() validation

After

```
// NotebookBuilder.java
public class NotebookBuilder {
    private String title;
    private String content = "";
```

```
    private Category category;
    private User owner;

    public NotebookBuilder title(String title)         { this.title = title;
return this; }
    public NotebookBuilder content(String content)     { this.content = content
!= null ? content : ""; return this; }
    public NotebookBuilder category(Category category) { this.category =
category; return this; }
    public NotebookBuilder owner(User owner)           { this.owner = owner;
return this; }

    public Notebook build() {
        if (owner == null) throw new IllegalStateException("owner must be set
before build()");
        Notebook notebook = new Notebook();
        notebook.setTitle(title);
        notebook.setContent(content);
        notebook.setCategory(category);
        notebook.setUser(owner);
        return notebook;
    }
}

// AFTER — NotebookService.createNotebook()
Notebook notebook = new NotebookBuilder()
        .title(request.getTitle())
        .content(request.getContent())
        .category(category)
        .owner(userService.findById(userId))
        .build();
```

Justification

The Builder pattern was chosen because Notebook has a mix of required (user) and optional (category, content) fields. The builder makes this distinction explicit in code rather than relying on JPA constraints to catch it at persist time, which shifts the error earlier and produces a clearer message.

Improvement Brought

- Construction and business logic are now clearly separated within the service method.
- Required field enforcement happens at build(), not at the database.
- Any new notebook creation site can reuse NotebookBuilder without copying null-guard logic.

Refactor 3 — Adapter on AI Provider

Original Implementation

AiService held a direct dependency on ProxyProvider — a concrete class that made raw HTTP calls to an OpenAI-compatible proxy. There was no interface between them.

```
// BEFORE — AiService.java
private final ProxyProvider proxyProvider;

String aiMessage = proxyProvider.generateResponse(
    config.getProxyUrl(), apiKey, config.getModel(), messages, 0.4
);
```

Problems with the Original

- Tight coupling to HTTP implementation: AiService was bound to ProxyProvider's concrete class.
- Untestable without HTTP: Unit tests required a real ProxyProvider making HTTP calls.
- No documented contract: No interface documenting what an AI provider must support.

Applied Design Pattern

Adapter: AiProvider target interface; ProxyProvider as the concrete adapter

After

```
// AiProvider.java — Target interface
public interface AiProvider {
    String generateResponse(String proxyUrl, String apiKey, String model,
                            List<Map<String, String>> messages, double
temperature);
    SpeechTranscriptionResponse transcribeAudio(String proxyUrl, String apiKey,
                                                MultipartFile file, String
language);
}

// ProxyProvider.java — Concrete Adapter
@Component @RequiredArgsConstructor
public class ProxyProvider implements AiProvider {
    @Override
    public String generateResponse(...) { /* HTTP implementation */ }
    @Override
    public SpeechTranscriptionResponse transcribeAudio(...) { /* multipart HTTP
*/ }
}

// AiService.java — now depends on the interface
private final AiProvider proxyProvider;
```

Justification

The Adapter pattern was chosen because ProxyProvider wraps a third-party communication mechanism and translates it into the format BrainBox needs. AiProvider is the target contract. AiService is now decoupled from HTTP entirely — it works with any AiProvider implementation.

Improvement Brought

- AiService is now independently unit-testable by injecting a mock AiProvider.

- The AI backend can be swapped (direct API, local model, mock) with no changes to AiService.
- The AiProvider interface serves as explicit documentation of what any AI backend must implement.

Refactor 4 — Observer on Notebook Versioning

Original Implementation

NotebookService directly called NotebookVersionSnapshotService.createSnapshot() in two places: after creating a notebook and after saving content.

```
// BEFORE — NotebookService.java
private final NotebookVersionSnapshotService notebookVersionSnapshotService;

// In createNotebook() and saveContent():
Notebook savedNotebook = notebookRepository.saveAndFlush(notebook);
notebookVersionSnapshotService.createSnapshot(savedNotebook,
savedNotebook.getContent());
```

Problems with the Original

- Coupled concerns: Notebook persistence and version snapshotting are separate concerns, but NotebookService triggered both.
- Hard to extend: Adding a second post-save reaction required modifying NotebookService again.
- Hidden dependency: NotebookService depended on NotebookVersionSnapshotService only to trigger a side effect.

Applied Design Pattern

Observer: Spring ApplicationEvent as the event; NotebookVersionSnapshotListener as the observer

After

```
// NotebookContentSavedEvent.java
public class NotebookContentSavedEvent extends ApplicationEvent {
    private final Notebook notebook;
    private final String content;
    // constructor ...
}

// NotebookVersionSnapshotListener.java
@Component @RequiredArgsConstructor
public class NotebookVersionSnapshotListener {
    private final NotebookVersionSnapshotService
notebookVersionSnapshotService;
    @EventListener
    public void onNotebookContentSaved(NotebookContentSavedEvent event) {
        notebookVersionSnapshotService.createSnapshot(event.getNotebook(),
event.getContent());
```

```
    }
}

// AFTER — NotebookService.java
private final ApplicationEventPublisher eventPublisher;

Notebook savedNotebook = notebookRepository.saveAndFlush(notebook);
eventPublisher.publishEvent(
    new NotebookContentSavedEvent(this, savedNotebook,
savedNotebook.getContent())
);
```

Justification

The Observer pattern was chosen because the versioning reaction is a side effect of saving — not part of NotebookService's core responsibility. Spring's ApplicationEventPublisher is the framework-native way to implement this pattern. The pattern perfectly fits the 'one action triggers many reactions' scenario without any of the components knowing about each other.

Improvement Brought

- NotebookService no longer depends on NotebookVersionSnapshotService.
- Future observers (analytics, search indexing) can be added as new @EventListener beans with zero changes to NotebookService.
- Each observer is independently testable by publishing the event directly.

Refactor 5 — Template Method on AI Prompt Construction

Original Implementation

AiService contained a buildSystemPrompt() private method spanning over 180 lines of String.format() with %s placeholders, plus seven build*Block() helper methods. A reviewMode boolean was passed through the method chain to switch one section of the prompt.

```
// BEFORE — AiService.java (simplified)
private String buildSystemPrompt(
    String notebookTitle, String context, String selectedText,
    String selectionMode, List<AiSelectionTarget> aiSelections,
    boolean reviewMode   // threaded through every helper
) {
    return String.format(
        "... %s ... %s ... %s ...",
        notebookTitle,
        buildWorkingScopeBlock(), context,
        buildReviewModeBlock(reviewMode),  // only step that varies
        buildFormattingGuidanceBlock()
    );
}
```

Problems with the Original

- Giant method: 180+ lines in a single private method obscured what differed between modes.
- Boolean flag threading: reviewMode was passed through buildSystemPrompt() and into helper methods (classic flag argument smell).
- Difficult to extend: Adding a third mode required modifying the existing method rather than adding a new class.
- SRP violation: Prompt construction and AI response handling co-located in AiService.

Applied Design Pattern

Template Method: AiPromptBuilder abstract class; EditorModePromptBuilder and ReviewModePromptBuilder as concrete subclasses

After

```
// AiPromptBuilder.java — abstract base (invariant skeleton)
public abstract class AiPromptBuilder {
    public final String buildSystemPrompt() {
        return String.format("...",
            notebookTitle,
            buildWorkingScopeBlock(),   // invariant
            buildModeBlock(),           // HOOK — subclass overrides
            buildFormattingGuidanceBlock()  // invariant
        );
    }
    protected abstract String buildModeBlock();  // hook method
}

// EditorModePromptBuilder.java
public class EditorModePromptBuilder extends AiPromptBuilder {
    @Override
    protected String buildModeBlock() {
        return "ASSISTANT MODE: editor\n\nYou may use editor actions ...\n\n";
    }
}

// ReviewModePromptBuilder.java
public class ReviewModePromptBuilder extends AiPromptBuilder {
    @Override
    protected String buildModeBlock() {
        return "ASSISTANT MODE: review\n\n- Only: none, create_quiz,
create_flashcard.\n\n";
    }
}

// AiService.java — usage
AiPromptBuilder promptBuilder = reviewMode
    ? new ReviewModePromptBuilder(title, context, selectedText, selectionMode,
aiSelections)
    : new EditorModePromptBuilder(title, context, selectedText, selectionMode,
aiSelections);
String systemPrompt = promptBuilder.buildSystemPrompt();
```

Justification

The Template Method pattern was the right choice because the prompt construction algorithm has a fixed skeleton with exactly one step that varies per mode. Placing the skeleton in a final method in the base class prevents accidental reordering of sections while allowing safe customization of the one variable step. This is the canonical use case for Template Method.

Improvement Brought

- AiService.generateResponse() no longer contains any prompt text — it selects a builder and calls one method.
- The reviewMode boolean parameter is eliminated from the call chain.
- Adding a new mode requires creating one new subclass, not editing existing code.
- Prompt construction logic is independently testable without instantiating AiService.

Refactor 6 — Facade on Authentication Layer

Original Implementation

AuthController imported and depended on AuthService — the concrete class that orchestrates JWTService, UserService, CodeService, RefreshTokenService, and EmailService.

```
// BEFORE — AuthController.java
import edu.cit.gako.brainbox.auth.service.AuthService;

@RestController @RequiredArgsConstructor
public class AuthController {
    private final AuthService authService; // concrete class

    @PostMapping("/login")
    public ResponseEntity<ApiResponse<LoginResponse>> login(...) {
        return ResponseEntity.ok(ApiResponse.success(
            authService.login(request, servletRequest)));
    }
}
```

Problems with the Original

- Controller coupled to service implementation: AuthController had to import and know about AuthService directly.
- Testing required a full AuthService: No interface to mock, so integration testing required the entire auth subsystem.
- Subsystem complexity was invisible: No contract documenting which operations the auth layer exposed.
- Accidental API surface: Other classes could directly inject AuthService, bypassing intended access patterns.

Applied Design Pattern

Facade: AuthFacade interface; AuthService as the concrete facade

After

```
// AuthFacade.java — the facade interface
public interface AuthFacade {
    void register(RegisterRequest request);
    void verifyEmail(String token);
    void forgotPassword(String email);
    VerifyCodeResponse verifyCode(String email, String code);
    void resetPassword(String token, String newPassword);
    LoginResponse login(LoginRequest request, HttpServletRequest
servletRequest);
    void logout(LogoutRequest request);
    LoginResponse refreshToken(String refreshToken);
    LoginResponse googleLogin(String accessToken, HttpServletRequest
servletRequest);
}

// AuthService.java — implements the facade
@Service @RequiredArgsConstructor
public class AuthService implements AuthFacade {
    // orchestrates JWTService, UserService, CodeService,
    // RefreshTokenService, EmailService behind the interface
}

// AuthController.java — depends on the interface
import edu.cit.gako.brainbox.auth.service.AuthFacade;

@RestController @RequiredArgsConstructor
public class AuthController {
    private final AuthFacade authService; // interface only
}
```

Justification

AuthService was already acting as a facade informally — it was the single point that coordinated five services behind meaningful operations. Formalizing it with AuthFacade makes the pattern explicit, documents the public contract, and allows the controller to be tested with a mock. This is also the cleanest way to ensure the controller never reaches into subsystem services directly.

Improvement Brought

- AuthController is decoupled from the concrete AuthService implementation.
- The AuthFacade interface serves as a clear, explicit contract of what the auth system exposes to the HTTP layer.
- The controller can be unit-tested by injecting a mock AuthFacade — no JWT, no database, no email required.
- The subsystem is hidden: controllers cannot accidentally call JWTService or CodeService directly.

Summary of All Applied Patterns

#	Pattern	Category	File(s) Changed	Key Benefit
1a	**Strategy**	Behavioral	EmailSender, SmtpEmailSender, ResendEmailSender	Each delivery mechanism isolated, independently testable
1b	**Factory Method**	Creational	EmailSenderFactory	Strategy selected once at startup, not on every call
2	**Builder**	Creational	NotebookBuilder, NotebookService	Required-field enforcement, fluent construction
3	**Adapter**	Structural	AiProvider, ProxyProvider, AiService	HTTP implementation swappable, service independently testable
4	**Observer**	Behavioral	NotebookContentSavedEvent, NotebookVersionSnapshotListener, NotebookService	Save and versioning fully decoupled
5	**Template Method**	Behavioral	AiPromptBuilder, EditorModePromptBuilder, ReviewModePromptBuilder, AiService	Mode flag eliminated, prompt skeleton protected
6	**Facade**	Structural	AuthFacade, AuthService, AuthController	Controller decoupled from implementation, testable with mock